

مستندات فایل polyp_dataset.py

کلاس Dataset و توابع سازنده برای پروژه سگمنتیشن پولیپ

پایان نامه کارشناسی ارشد - مهندسی کامپیوتر، گرایش هوش مصنوعی

۷ مهر ۱۴۰۵

فهرست مطالب

۲	۱	مقدمه
۲	۲	بخش ۱: Docstring فایل
۲	۳	بخش ۲: ساختار JSON مورد انتظار
۳	۴	بخش ۳: کلاس PolypDataset
۳	۱.۴	امضای کلاس و ارث‌بری
۳	۲.۴	متد __init__
۵	۳.۴	متد generate_data_list
۵	۱.۳.۴	مرحله ۱: خواندن JSON
۵	۲.۳.۴	مرحله ۲: انتخاب بخش مناسب
۵	۳.۳.۴	مرحله ۳: فیلتر کردن test (اختیاری)
۶	۴.۳.۴	مرحله ۴: خطای واضح در صورت نبود نمونه
۶	۵.۳.۴	مرحله ۵: خطا در section نامعتبر
۶	۴.۴	متد __repr__
۷	۵	بخش ۴: تابع get_polyp_dataset
۷	۱.۵	هدف
۸	۲.۵	تحلیل مرحله به مرحله
۸	۳.۵	نکته‌ی مهم: val در cache_rate
۹	۶	بخش ۵: تابع get_test_polyp_dataset
۹	۱.۶	هدف
۹	۲.۶	تحلیل مرحله به مرحله
۹	۳.۶	نمونه‌های استفاده
۹	۱.۳.۶	مثال ۱: کل test
۱۰	۲.۳.۶	مثال ۲: فقط Kvasir-SEG
۱۰	۳.۳.۶	مثال ۳: حلقه روی ۵ دیتاست
۱۰	۷	بخش ۶: جریان کامل داده - نمودار
۱۱	۸	بخش ۷: مصرف حافظه و عملکرد
۱۱	۱.۸	مصرف RAM برای cache
۱۱	۲.۸	سرعت
۱۱	۹	بخش ۸: جمع‌بندی
۱۱	۱.۹	خلاصه‌ی کلاس PolypDataset
۱۲	۲.۹	خلاصه‌ی توابع
۱۲	۳.۹	نکات کلیدی

۱ مقدمه

فایل polyp_dataset.py مسئول تعریف کلاس **Dataset** و توابع سازنده برای پروژه سگمنتیشن پولیپ است. این فایل از سه بخش اصلی تشکیل شده است:

۱. کلاس **PolypDataset**: یک Dataset سفارشی که از CacheDataset کتابخانه MONAI ارث‌بری می‌کند.

۲. تابع **get_polyp_dataset**: ساخت دیتاست آموزش و اعتبارسنجی.

۳. تابع **get_test_polyp_dataset**: ساخت دیتاست تست با پشتیبانی از فیلتر کردن بر اساس نام دیتاست.

نقش فایل در پروژه: این فایل، پل بین فایل **split.json** و حلقه‌ی آموزش است. داده‌ها را از JSON می‌خواند، ترنسفورم‌ها را روی آن‌ها اعمال می‌کند، و نتیجه را به صورت یک Dataset آماده برای DataLoader تحویل می‌دهد.

۲ بخش ۱: Docstring فایل

```
1 """
2 Polyp Dataset
3 =====
4
5 This module defines the PolypDataset class for polyp segmentation.
6
7 PolypDataset is built on top of MONAI's CacheDataset and reads its list
8 of samples from a JSON split file. Supported sections:
9
10     - 'train'      : uses data['fold{N}']['train']
11     - 'validation' : uses data['fold{N}']['val']
12     - 'test'       : uses data['test'] (at root level)
13
14 The 'test' section supports optional filtering by 'dataset_origin'
15 (e.g., 'Kvasir-SEG', 'CVC-ClinicDB', 'CVC-300', ...) for per-dataset
16 evaluation.
17 """
```

توضیح: این docstring بالای فایل، خلاصه‌ای از هدف ماژول و ساختار JSON مورد انتظار را ارائه می‌دهد. این مهم است چون:

- هدف را روشن می‌کند: ساخت Dataset برای polyp.
- ساختار JSON را مستند می‌کند.
- امکان فیلتر کردن test بر اساس dataset_origin را نشان می‌دهد.

۳ بخش ۲: ساختار JSON مورد انتظار

```
1 {
2   "fold0": {
3     "train": [
4       {"image": "...", "mask": "...", "dataset_origin": "..."},
5       ...
6     ],
7     "val": [
8       {"image": "...", "mask": "...", "dataset_origin": "..."},
9       ...
10    ]
11  },
12  "test": [
13    {"image": "...", "mask": "...", "dataset_origin": "...",
```

```

۱۴     "distribution": "..."},
۱۵     ...
۱۶ ]
۱۷ }

```

تحلیل ساختار:

جدول ۱: ساختار JSON در هر بخش

بخش	مکان	تعداد (polyp)	کلیدها
train	fold0.train	۱۳۰۵	dataset_origin, mask, image
validation	fold0.val	۱۴۵	dataset_origin, mask, image
test	root.test	۷۹۸	distribution, dataset_origin, mask, image

نکته‌ی مهم: test در سطح ریشه است (نه زیر fold0) چون تست polyp از دیتاست آماده‌ی TestDataset می‌آید و به fold وابسته نیست.

۴ بخش ۳: کلاس PolypDataset

۱.۴ امضای کلاس و ارث‌بری

```

۱ class PolypDataset(CacheDataset):

```

توضیح:

- کلاس PolypDataset از CacheDataset (کتابخانه MONAI) ارث‌بری می‌کند.

- CacheDataset به صورت خودکار:

- ترنسفورم‌های غیر-تصادفی (مثل LoadImageD، NormalizeIntensityD) را یک بار اجرا و cache می‌کند.

- ترنسفورم‌های تصادفی (مثل RandFlipD) را در هر **__getitem__** دوباره اعمال می‌کند.

مزیت بزرگ: با استفاده از CacheDataset، هم از سرعت cache بهره‌مند می‌شویم و هم تنوع augmentation حفظ می‌شود.

۲.۴ متد __init__

```

۱ def __init__(
۲     self,
۳     split_path: str,
۴     section: str,
۵     num_fold: int,
۶     transforms,
۷     dataset_origin: Optional[str] = None,
۸     cache_num: int = sys.maxsize,
۹     cache_rate: float = 0.0,
۱۰    num_workers: int = 0,
۱۱ ):
۱۲     self.section = section
۱۳     self.num_fold = num_fold
۱۴     self.dataset_origin = dataset_origin
۱۵

```

```

۱۶ data = self._generate_data_list(split_path)
۱۷ self.num_samples = len(data)
۱۸
۱۹ super().__init__(
۲۰     data,
۲۱     transforms,
۲۲     cache_num=cache_num,
۲۳     cache_rate=cache_rate,
۲۴     num_workers=num_workers,
۲۵ )

```

توضیح پارامترها:

پارامتر	توضیح
split_path	مسیر فایل JSON تعریف کننده ی train/val/test.
section	یکی از 'train'، 'validation'، 'test'.
num_fold	شماره fold (فقط برای train و validation).
transforms	pipeline ترکیبی از Compose که روی هر نمونه اعمال می شود.
dataset_origin	اگر داده شود و section='test' باشد، فقط نمونه های مربوط به این دیتاست نگه داشته می شوند.
cache_num	حداکثر تعداد آیتم برای cache در RAM.
cache_rate	نسبت داده ی cache شده (۰.۰ تا ۰.۱).
num_workers	تعداد worker برای بارگذاری موازی.

تحلیل مرحله به مرحله:

۱. ذخیره ی attribute ها:

```

۱ self.section = section
۲ self.num_fold = num_fold
۳ self.dataset_origin = dataset_origin

```

این ها برای استفاده در generate_data_list و __repr__ ذخیره می شوند.

۲. خواندن لیست نمونه ها:

```

۱ data = self._generate_data_list(split_path)

```

متد generate_data_list را صدا می زند تا لیست نمونه ها را از JSON استخراج کند.

۳. ذخیره ی تعداد نمونه:

```

۱ self.num_samples = len(data)

```

برای استفاده در __repr__.

۴. فراخوانی سازنده ی پدر:

```

۱ super().__init__(data, transforms, cache_num=..., cache_rate=..., num_workers=...)

```

لیست داده + transforms را به CacheDataset پاس می دهد.

۳.۴ متد generate_data_list

```
1 def _generate_data_list(self, split_path: str):
2     with open(split_path) as fp:
3         data = json.load(fp)
4
5     if self.section == 'train':
6         datalist = data[f'fold{self.num_fold}']['train']
7
8     elif self.section == 'validation':
9         datalist = data[f'fold{self.num_fold}']['val']
10
11    elif self.section == 'test':
12        datalist = data['test']
13        if self.dataset_origin is not None:
14            datalist = [
15                sample for sample in datalist
16                if sample.get('dataset_origin') == self.dataset_origin
17            ]
18        if not datalist:
19            available = sorted({
20                s.get('dataset_origin') for s in data['test']
21            })
22            raise ValueError(
23                f"No samples found for dataset_origin="
24                f"'{self.dataset_origin}'. "
25                f"Available origins: {available}"
26            )
27
28    else:
29        raise ValueError(
30            f"Unsupported section: '{self.section}'. "
31            f"Available options are ['train', 'validation', 'test']. "
32        )
33
34    return datalist
```

تحلیل مرحله به مرحله:

۱.۳.۴ مرحله ۱: خواندن JSON

```
1 with open(split_path) as fp:
2     data = json.load(fp)
```

- کل فایل JSON در RAM بارگذاری می‌شود.
- حجم: تقریباً ۶۰۰ KB (برای polyp).
- نکته: بعد از return، متغیر data آزاد می‌شود.

۲.۳.۴ مرحله ۲: انتخاب بخش مناسب

۳.۳.۴ مرحله ۳: فیلتر کردن test (اختیاری)

```
1 if self.dataset_origin is not None:
2     datalist = [
3         sample for sample in datalist
4         if sample.get('dataset_origin') == self.dataset_origin
5     ]
```

توضیح:

جدول ۲: کلیدهای JSON برای هر section

section	کلید JSON	تعداد polyp
'train'	data['fold0']['train']	۱۳۰۵
'validation'	data['fold0']['val']	۱۴۵
'test'	data['test']	۷۹۸

- اگر dataset_origin داده شود، فقط نمونه‌های مربوط به آن دیتاست نگه داشته می‌شوند.
- sample.get('dataset_origin') به جای sample['...'] استفاده شده برای ایمنی (اگر کلید نبود، None برمی‌گردد).

مقادیر ممکن برای polyp:

dataset_origin	تعداد	دسته
Kvasir-SEG	۱۰۰	in_distribution
CVC-ClinicDB	۶۲	in_distribution
CVC-300	۶۰	out_of_distribution
CVC-ColonDB	۳۸۰	out_of_distribution
ETIS-LaribPolypDB	۱۹۶	out_of_distribution

۴.۳.۴ مرحله‌ی ۴: خطای واضح در صورت نبود نمونه

```

۱ if not datalist:
۲     available = sorted({
۳         s.get('dataset_origin') for s in data['test']
۴     })
۵     raise ValueError(
۶         f"No samples found for dataset_origin='{self.dataset_origin}'. "
۷         f"Available origins: {available}"
۸     )

```

مزیت: اگر کاربر dataset_origin را اشتباه بنویسد (مثلاً 'Kvasir' به جای 'Kvasir-SEG')، خطای واضح با لیست گزینه‌های موجود می‌گیرد.

۵.۳.۴ مرحله‌ی ۵: خطا در section نامعتبر

```

۱ else:
۲     raise ValueError(
۳         f"Unsupported section: '{self.section}'. "
۴         f"Available options are ['train', 'validation', 'test']."
۵     )

```

اگر section یکی از سه مقدار مجاز نباشد، ValueError پرتاب می‌شود.

۴.۴ متد __repr__

```

1 def __repr__(self) -> str:
2     parts = [
3         f"section='{self.section}'",
4         f"num_fold={self.num_fold}",
5     ]
6     if self.dataset_origin is not None:
7         parts.append(f"dataset_origin='{self.dataset_origin}'")
8     parts.append(f"num_samples={getattr(self, 'num_samples', '?')}")
9     parts.append(f"cache_rate={self.cache_rate}")
10    return f"PolypDataset({'', '.join(parts)})"

```

هدف: نمایش خوانا از وضعیت Dataset در Jupyter/Kaggle
مثال‌های خروجی:

سناریو	خروجی __repr__
train	PolypDataset(section='train', num_fold=0, num_samples=1305, cache_rate=0.3)
test (کامل)	PolypDataset(section='test', num_fold=0, num_samples=798, cache_rate=0.0)
test (Kvasir-SEG)	PolypDataset(section='test', num_fold=0, dataset_origin='Kvasir-SEG', num_samples=100, cache_rate=0.0)

۵ بخش ۴: تابع get_polyp_dataset

```

1 def get_polyp_dataset(args, sam_image_dim=None):
2     split_path = args['split_path']
3     num_fold = args['n_fold']
4     cache_rate = args.get('cache_rate', 0.0)
5
6     train_transforms, val_transforms = get_polyp_transforms(
7         args, sam_image_dim
8     )
9
10    ds_train = PolypDataset(
11        split_path=split_path,
12        section='train',
13        num_fold=num_fold,
14        transforms=train_transforms,
15        cache_rate=cache_rate,
16    )
17    ds_val = PolypDataset(
18        split_path=split_path,
19        section='validation',
20        num_fold=num_fold,
21        transforms=val_transforms,
22        cache_rate=cache_rate,
23    )
24    return ds_train, ds_val

```

۱.۵ هدف

ساخت دو دیتاست (آموزش و اعتبارسنجی) به صورت همزمان.

۲.۵ تحلیل مرحله به مرحله

۱. استخراج پارامترها از **args**:

```
۱ split_path = args['split_path']
۲ num_fold = args['n_fold']
۳ cache_rate = args.get('cache_rate', 0.0)
```

args.get('cache_rate', 0.0) یعنی اگر cache_rate در args نبود، مقدار پیش فرض ۰.۰ استفاده شود (ایمن تر از args['...']).

۲. ساخت ترنسفورمها:

```
۱ train_transforms, val_transforms = get_polyp_transforms(args, sam_image_dim)
```

از polyp_transforms.py، دو pipeline جداگانه ساخته می شود:

- train_transforms با augmentation.
- val_transforms بدون augmentation.

۳. ساخت **Dataset** آموزش:

```
۱ ds_train = PolypDataset(
۲     split_path=split_path,
۳     section='train',
۴     num_fold=num_fold,
۵     transforms=train_transforms,
۶     cache_rate=cache_rate,
۷ )
```

۴. ساخت **Dataset** اعتبارسنجی:

```
۱ ds_val = PolypDataset(
۲     split_path=split_path,
۳     section='validation',
۴     num_fold=num_fold,
۵     transforms=val_transforms,
۶     cache_rate=cache_rate,
۷ )
```

۵. بازگشت:

```
۱ return ds_train, ds_val
```

۳.۵ نکته ی مهم: cache_rate در val

در این تابع، cache_rate برای ds_val هم اعمال می شود. اگر cache_rate بالا باشد (مثلاً ۰.۱)، ds_val هم به RAM می رود و حافظه ی بیشتری مصرف می کند. راه حل اختیاری: برای val، cache_rate را به ۰.۰ تنظیم کنید (چون val فقط یک بار در هر epoch خوانده می شود):

```
۱ ds_val = PolypDataset(
۲     ...
۳     cache_rate=0.0, # برای صرفه جویی در RAM
۴ )
```

۶ بخش ۵: تابع `get_test_polyp_dataset`

```
1 def get_test_polyp_dataset(args, sam_image_dim=None, dataset_origin=None):
2     split_path = args['split_path']
3     num_fold = args['n_fold']
4     cache_rate = args.get('cache_rate', 0.0)
5
6     _, val_test_transforms = get_polyp_transforms(args, sam_image_dim)
7
8     ds_test = PolypDataset(
9         split_path=split_path,
10        section='test',
11        num_fold=num_fold,
12        transforms=val_test_transforms,
13        dataset_origin=dataset_origin,
14        cache_rate=cache_rate,
15    )
16    return ds_test
```

۱.۶ هدف

ساخت یک دیتاست تست که می‌تواند:

- کل ۷۹۸ نمونه را شامل شود (اگر `dataset_origin=None`).
- یک دیتاست خاص را شامل شود (اگر `dataset_origin` داده شود).

۲.۶ تحلیل مرحله به مرحله

۱. استخراج پارامترها: مشابه `get_polyp_dataset`.

۲. ساخت ترنسفورمها:

```
1 _, val_test_transforms = get_polyp_transforms(args, sam_image_dim)
```

نکته: از `_` استفاده شده چون `train_transforms` لازم نیست. فقط `val_test_transforms` مورد استفاده قرار می‌گیرد.

۳. ساخت Dataset تست:

```
1 ds_test = PolypDataset(
2     split_path=split_path,
3     section='test',
4     num_fold=num_fold,
5     transforms=val_test_transforms,
6     dataset_origin=dataset_origin, # اینجا معملاً می‌شود
7     cache_rate=cache_rate,
8 )
```

۴. بازگشت:

```
1 return ds_test
```

۳.۶ نمونه‌های استفاده

۱.۳.۶ مثال ۱: کل test

```
1 ds_test = get_test_polyp_dataset(args, sam_image_dim=1024)
2 print(ds_test)
3 # PolypDataset(section='test', num_fold=0, num_samples=798, cache_rate=0.0)
```

۲.۳.۶ مثال ۲: فقط Kvasir-SEG

```

۱ ds_kvasir = get_test_polyp_dataset(
۲     args, sam_image_dim=1024, dataset_origin='Kvasir-SEG'
۳ )
۴ print(ds_kvasir)
۵ # PolypDataset(section='test', num_fold=0, dataset_origin='Kvasir-SEG',
۶ #               num_samples=100, cache_rate=0.0)

```

۳.۳.۶ مثال ۳: حلقه روی ۵ دیتاست

```

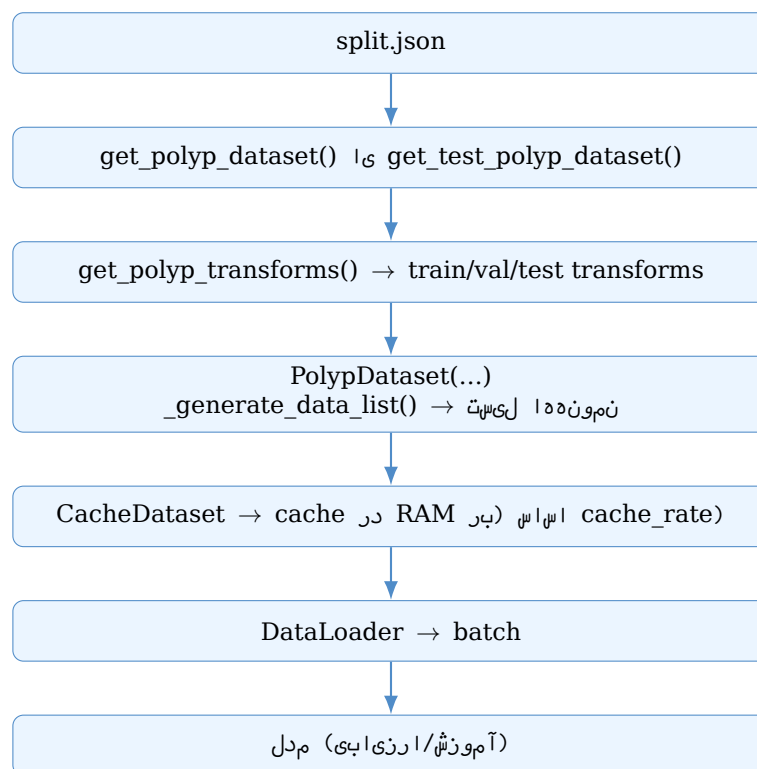
۱ DATASETS = ['Kvasir-SEG', 'CVC-ClinicDB', 'CVC-300',
۲             'CVC-ColonDB', 'ETIS-LaribPolypDB']
۳
۴ for name in DATASETS:
۵     ds = get_test_polyp_dataset(args, img_dim, dataset_origin=name)
۶     loader = DataLoader(ds, batch_size=1, shuffle=False, num_workers=0)
۷     metrics = evaluate(model, loader)
۸     print(f"{name:25s}: Dice={metrics['dice']:.4f}")

```

خروجی مورد انتظار:

Kvasir-SEG	: Dice=0.9182
CVC-ClinicDB	: Dice=0.9251
CVC-300	: Dice=0.8743
CVC-ColonDB	: Dice=0.7824
ETIS-LaribPolypDB	: Dice=0.7862

۷ بخش ۶: جریان کامل داده – نمودار



شکل ۱: جریان کامل داده از JSON تا مدل

۸ بخش ۷: مصرف حافظه و عملکرد

۱.۸ مصرف RAM برای cache

جدول ۳: مصرف RAM بر اساس cache_rate (برای train با ۱۳۰۵ نمونه)

cache_rate	تعداد نمونه‌ی cache	مصرف RAM
۰.۰	۰	۰~
۱.۰	۱۳۰	GB ۱۸.۲~
۳.۰	۳۹۱	GB ۵۶.۶~
۵.۰	۶۵۲	GB ۹.۱۰~
۰.۱	۱۳۰۵	GB ۹.۲۱~

نکته برای Colab/Kaggle:

- Colab Free (GB ۷.۱۲): cache_rate=0.1 توصیه می‌شود.
- Kaggle (GB ۱۶): cache_rate=0.2 توصیه می‌شود.
- Colab Pro (GB ۲۵): cache_rate=0.3 توصیه می‌شود.

۲.۸ سرعت

جدول ۴: سرعت CacheDataset با cache_rate مختلف (epoch ۱۵۰)

cache_rate	زمان هر epoch	زمان کل (epoch ۱۵۰)
۰.۰	~۲۶ s	~۶۵ min
۱.۰	~۲۳ s	~۵۸ min
۳.۰	~۱۸ s	~۴۵ min
۰.۱	~۱.۰ s	~۸ sec

۹ بخش ۸: جمع‌بندی

۱.۹ خلاصه‌ی کلاس PolypDataset

- ارث‌بری: از CacheDataset کتابخانه MONAI.
- منطق: خواندن از JSON + cache کردن + پشتیبانی از augmentation.
- ویژگی‌ها:
 - سه section: train, validation, test.
 - فیلتر dataset_origin برای test.
 - خطای واضح در صورت section یا dataset_origin نامعتبر.
 - __repr__ برای دیباگ.

تابع	کاربرد	خروجی
get_polyp_dataset	ساخت train و val	(ds_train, ds_val)
get_test_polyp_dataset	ساخت test (با فیلتر اختیاری)	ds_test

۵ نکته‌ی طلایی:

۱. CacheDataset هم cache می‌کند و هم augmentation را حفظ می‌کند.
۲. cache_rate مناسب برای Colab Free: ۱.۰.
۳. dataset_origin امکان گزارش per-dataset را فراهم می‌کند.
۴. sample.get('dataset_origin') ایمن‌تر از sample['...'] است.
۵. خطای واضح در dataset_origin اشتباه، زمان دیباگ را کم می‌کند.